

Applications of Parallel Computing

S. Afkhami

Applications of Parallel Computing

Copyright (C) 2023 S. Afkhami

Faculty Authored Open Textbook
New Jersey Institute of Technology
Newark, NJ, USA

Permission is granted to copy, distribute, transmit and adapt this work under the Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License: <https://creativecommons.org/licenses/by-nc/4.0/>.

If you are interested in distributing a commercial version of this work, please contact the author.

The L^AT_EX source and code for this book is available from
<https://github.com/afkhamis/HPC-book>

Preface

The goal of this book is to provide an introductory level overview of modern scientific computing. This book is intended to prepare students to be able to implement algorithmic techniques for high performance computing for applications. All the programs are developed in C programming language, which is the most widely used language for scientific computing.

The book consists of several parts that are developed to be independent of other parts. The book begins with an introduction to parallel computing. and then introducing the tools useful for developing the codes. This is followed by a crash course on C programming. It is assumed that the students have basic familiarity with the programming logic. Example codes are also provided.

Next, OpenMP is introduced. OpenMP provides a model for shared memory machines. OpenMP is the entry point to get started with parallel programming. It provides an easy to use platform for parallelization which is supported by C/C++ and Fortran on a wide variety of architectures. This part covers most of the major features of OpenMP, for example, its various constructs and directives for specifying parallel regions, work sharing, synchronization and data environment.

The Message Passing Interface Standard (MPI) is presented next. The goal of this part is to introduce the Message Passing Interface for writing message passing programs. MPI is the gold standard for writing message passing programs on HPC platforms. This part teaches students how to develop and run parallel MPI programs. This part begins with basic concepts for getting started with MPI, followed by a detailed look at the MPI routines that are widely used. A variety of examples are provided.

The next part addresses the application of GPU's in HPC. The fundamental tools and techniques for running C codes on GPUs with CUDA are introduced. CUDA parallelization, memory optimization, and migration between the CPU and GPU are shown. Students will encounter techniques that can be considered general parallel programming paradigms and will utilize CUDA streams for additional concurrency.

The selection of numerical methods was influenced by the practice. The finite difference method is commonly used for non-curved boundaries; here only a bare-bone discussion of it is introduced in the context of the solution of the elliptic partial differential equations. Simplest form of the Fast Fourier Transform and its parallelization is discussed. The goal of the Machine Learning part is to present the key concepts that form the core of machine learning. Machine learning has become an integral part of many modern applications. This part provides a view

to understand how to utilize algorithms for parallel Machine Learning.

INTRODUCTION TO PARALLEL COMPUTING

Parallel hardware has seen an increasing advances in recent decades. Nowadays, personal laptops and desktop use multicore processors. Computing clusters are essential for simulation in Science and Engineering and familiarity with parallel programming and architecture is a must for computer science majors. Real World problems are complex and without parallel computing simulations that are too big are too expensive to carry out. Massively large scale computations, particularly in Climate Modeling, Drug Discovery, and Data Analysis, rely on computer clusters comprising of thousands of processors. From this point view, it is important to understand the fundamentals, tools, and algorithms for parallel programming for currently available platforms. In what follows, an overview of parallel programming models are introduced with an emphasis on design of parallel programs illustrated with examples.

1.1 Serial Computing

Traditionally, codes were written for serial computation, i.e. running a single job at a time. In such case, a problem is divided into a series of instructions, which are consequently executed sequentially on a single processor. In this manner, instructions are execute one at a time. To understand this process further, it is best to first understand the structure of a sequential computer, which consists of main *memory*, a *central processing unit* (CPU), also called processor or core, that are connected via an *interconnection*. A memory consists of a collection of storage locations; both instructions and data are stored in the memory. A memory address points to the location of the memory where the contents, instructions or data, can be accessed. CPU consists of two units: Control Unit and Arithmetic Logic Unit. The Control Unit is for controlling the executing of instructions and

the Arithmetic and Logic Unit execute the instructions. Registers store the data and information about the execution in the CPU. This structure layout is known as a von Neumann machine and this separation of CPU and memory, which stores both program and data, is called the von Neumann bottleneck. This is because in the process of fetching the data and executing the instructions, instructions and data are transferred between the CPU and memory via the interconnect, and therefore the rate at which data can be transferred is limited by the interconnect. As an example, consider the following C code:

```
void add_store (double *a, double *b, double *c) {  
    *c = *a + *b;  
}
```

Here what happens:

- load **a** and **b** from memory to register
- load, combined with an arithmetic addition, to another register
- store back the sum result to memory location of **c**,

where compilers manage memory and registers. The cost of executing this routine includes the cost of the arithmetic operation and the memory accesses, load/store. Data transfers (load/store) have two costs:

- Latency: cost to load or store 1 word
- Bandwidth: average cost per byte to load/store a large chunk.

The performance of an interconnect depends on the latency and the bandwidth. The large chunk is typically more than one floating point number and therefore to increase performance many memory requests must be issued. However, due to latency, it will take time for the data to be transferred. To reach full bandwidth speed, a volume of independent data equal to the bandwidth times the latency has to be in flight, also known as the Little's law:

Concurrency = Bandwidth * Latency.

The overall performance is therefore determined not just by the processor performance, but also the speed of the memory system to transfer data. The clock speed of processors have increased at over years, while memory access times lag behind. In modern computer systems, this bottleneck is addressed by having a hierarchy of successively faster memory devices called caches. Cache is fast, but expensive, memory that takes advantage of the locality of the data. Many programs have a high degree of data locality; for example, when looping over an array, accessing

things nearby previous accesses, called spatial locality, or reusing an item that was previously accessed, called temporal locality. Accessing instructions or data item already residing in cache is cheap, called cache hit. If those not in cache, which is called a cache miss, then there will be a need to access next, slower level of memory. Computer hardware directly manages the cache memory. On-chip caches are faster, but limited in their sizes. Approaches for handling memory access can be viewed as:

- Eliminate memory operations by saving values in small, fast memory (cache or registers) and reusing them.
- Take advantage of better bandwidth by getting a chunk of memory into cache (or vector registers) and using whole chunk.
- Take advantage of better bandwidth by allowing processor to issue multiple loads/stores with a single instruction, or in parallel with other loads/stores/-operations.

Most programs run below maximum performance, mainly due to interconnect. Consider an example of computing inner product, involving one multiplication and addition per cycle. A processor operating at 1 GHz connected to a Dynamic Random-Access Memory (DRAM) with a latency of 100 ns with no caches. Considering two multiply-add units and executing 4 instructions per cycle, the peak performance is 4 GFLOPS. 100 ns latency means that at each cycle, processor waits 100 cycles before executing the next cycle, since each cycle is performed at 1 ns. On this platform, an inner product computation can be done with a speed of 10 MFLOP, which is much smaller than the peak performance. Next we review parallel computing to highlight the need for improving the performance loss on a single processor.

1.2 Parallel Computing Overview

In the previous section, we reviewed the concept of instructions can be executed simultaneously “in flight”, known as instruction-level parallelism, where users have no control over it. On the other hand, users can explicitly schedule more than one instruction to be handled by multiple processors. In the simplest sense, a problem is broken into multiple parts, and consequently further broken down to a series of instructions that can be executed concurrently on different processors. The control and coordination mechanism is employed by the programmer. In the following, we will review the hardware, the programming, and the concepts of this type of parallelism.

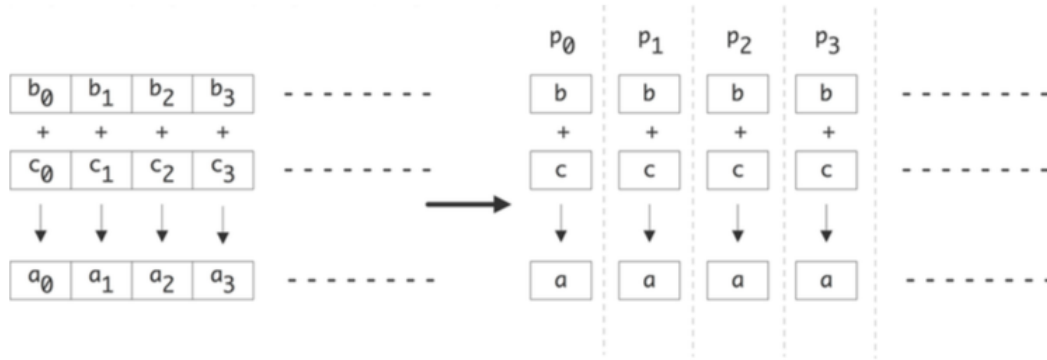
Consider a very simple example of adding two vectors:

```

for (i = 0; i < N; i++)
    a[i] = b[i] + c[i]

```

to be executed on N processors. Ideally, N processors can execute the single instruction $a = b + c$, as depicted below.



Ideally, this parallel algorithm should execute in one unit of time that takes to do one operation and in general, the execution time is linearly proportional to the number of processors. It is however not always simple to parallelize the problem. Often the algorithms must be redesigned to accommodate for parallelization. Even after that, the algorithm may not linearly reduce the execution with the number of processors.

There are a number of ways to classify computer structures:

- Single instruction stream, single data stream, or SISD, where only one instruction stream is being acted on by the processor and only one data stream is loaded at any one cycle.
- Single instruction, multiple data, or SIMD, where all processing units execute the same instruction at any given clock cycle and each processing unit can operate on a different data element.
- Multiple instruction, multiple data (MIMD), where every processor may be executing a different instruction stream and working with a different data stream.

An example of parallelism for SIMD architectures is the vector addition mentioned above, which can be modified as

```

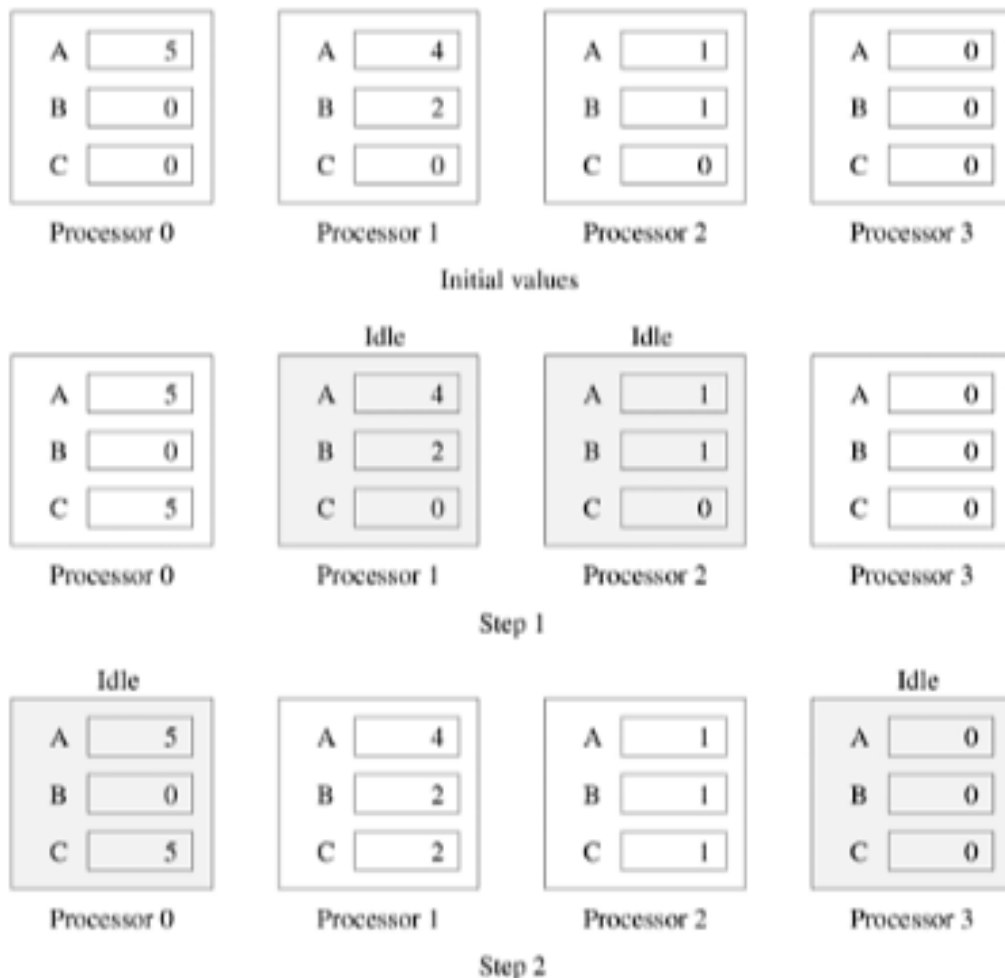
for (i = my_low; i < my_high; i++)
    a[i] = b[i] + c[i]

```

where each processor executes the same addition operation concurrently and syn-

chronously. SIMD architecture relies on the highly structured (regular) nature of the underlying computations, such as the vector addition above, and therefore ideal for parallelizing simple loops operating on huge arrays of data. The example above is called data-parallelism, where data is divided among the processors that each apply the same instructions to their data subsets. While SIMD computers are very efficient on large data parallel problems, they are not suitable for some parallelizations; for example, consider a conditional execution

```
if (B == 0)
    C = A;
else
    C = A/B;
```

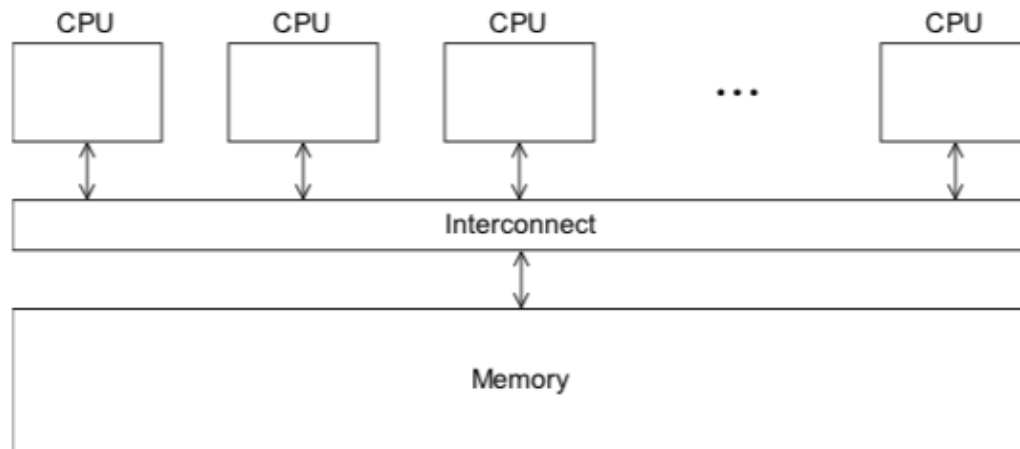


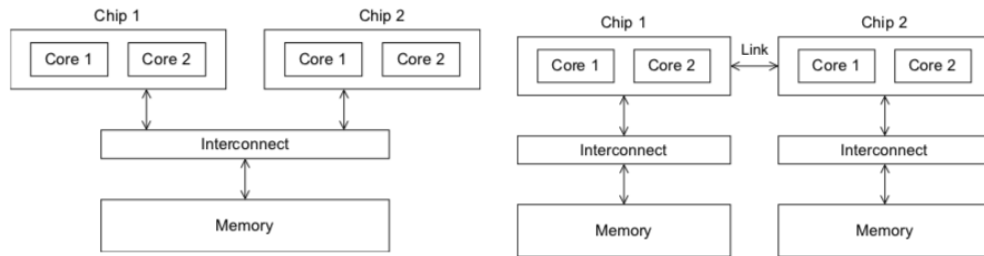
As shown, some processors become idle, leading to poor utilization of resources. Such drawbacks have made SIMD architecture less desirable and nowadays, the

most common parallel architecture is MIMD. We will mainly focus on MIMD machines that operate in Single Program Multiple Data (SPMD) mode, where the same executable act on their copy of the same program simultaneously that will not in general be synchronous as in SIMD machines. The lack of synchronization can be due to processors working on different amounts of data, known as load unbalance, leading to the loss of performance. There are two main types of SPMD models: shared-memory and distributed-memory models.

1.2.1 Shared Memory Model

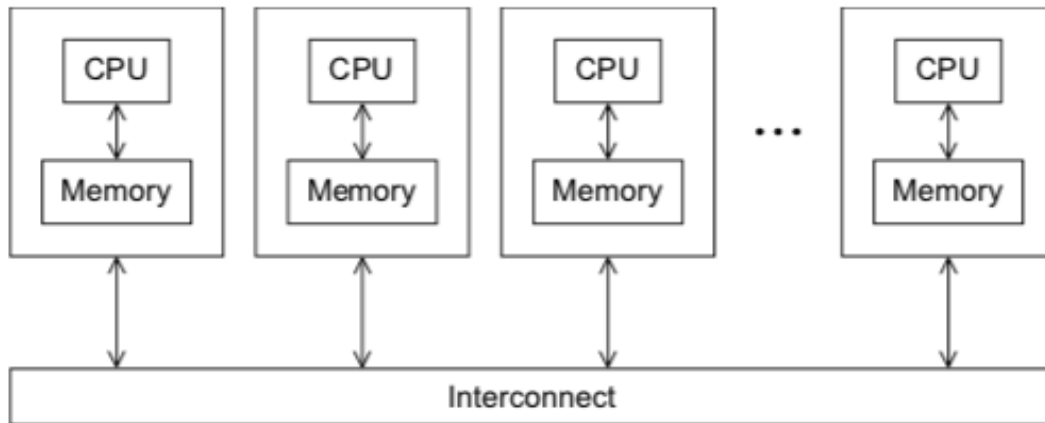
A shared-memory model is a parallel platform of processors that operate autonomously connected to a memory by an interconnection, and each processor has access to all memory locations. If the access time is the same for each processor, the system is called Uniform Memory Access (UMA), most commonly represented Symmetric Multi Processing (SMP) programming model. On the other hand, if the access time is different for certain processes, the system is called Non-Uniform Memory Access (NUMA). NUMA machines are often built by linking SMP machines, where one SMP can directly access memory of another one, however, memory location to which a processor is directly connected can be accessed more quickly than a memory location that must be accessed through the link. If and update to a location in shared memory done by one processor can be visible to all the other processors, then the system is called cache coherent. Below are diagrams of shared memory, UMA, and NUMA, systems:





1.2.2 Distributed Memory Model

Parallel programming is relatively straightforward if all processors have access to any memory location. Moreover, it provides data sharing between processors that is fast and uniform. However, the main disadvantage is the lack of scalability between memory and processors, due to increased traffic on the memory-processor interconnect, and for cache coherent systems, due to the cache-memory management. One solution is to have the processors have their own local memory and therefore each processor operates independently and any updates to its local memory is not visible to other processors. With this setup, the only way one processor can exchange information with another is through a communication network that connects inter-processor memory. This communication is set up by the programmer, as well as synchronization between tasks. Below is a diagram of a distributed memory system. The most widely used distributed memory machines are computer clusters and supercomputers. The interconnection network typically is Ethernet. Often, each computational unit, is a shared memory systems with multiprocessors, linked to other units through communication network. These are called hybrid systems. The individual units can be built from different types of hardware, making the system heterogeneous. Below is diagrams of a distributed memory systems:



1.3 Case study: Matrix Multiplication

Use of performance models to understand performance • Attainable lower bounds on communication • Simple cache model • Warm-up: Matrix-vector multiplication • Naive vs optimized Matrix-Matrix Multiply • Minimizing data movement • Beating $O(n^3)$ operations • Practical optimizations (continued next time)

PROGRAMMING TOOLS

Programming tools are needed to assist a programmer with basic aspects of programming. Programming environment consists of programming tools to facilitate programming and to enhance programming ease and productivity. Programming tools are needed for writing the actual code, editing existing code, to compile, debug and linking the code. In what follows, we review two distinct types of programming environments: Command-line and Integrated Development Environments.

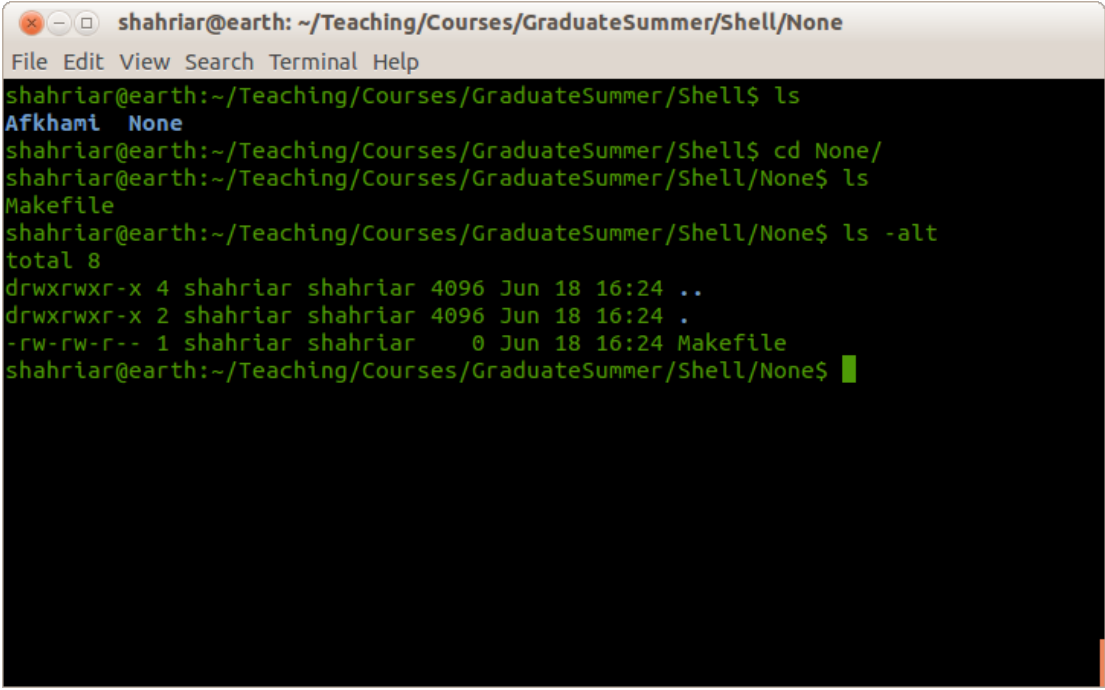
2.1 Command-line Environments

Command-line environment provides a collection of commands that can be used to edit codes, compile source codes, debug, and run programs. However, due to its nature, the beginners must to put in a some efforts to master the practice. The command-line environment can makes the use of scripting and batch files tremendously easy. The commonest example of this is the UNIX Shell. UNIX shells allows users to enter commands by providing a command-line interface. Typed in commands are translated by the shell into something the UNIX kernel can process. One of the widely used Shells available is `bash`.

Despite the fact that most modern tools have a graphical user interface (GUI), command-line user interfaces (CLUIs) still have their place; it is easier to see and understand what the computer is doing on your behalf and more importantly, it is easier to combine CLUI tools than GUI tools. In what follows, some basics of shell will be introduced. The command shell manages a user's interactions with the operating system by:

- Reading commands from the keyboard
- Figuring out what programs the user wants to run

- Running those programs
- Displaying their output on the screen
- Looks (and works) like an interactive terminal:



```
shahriar@earth: ~/Teaching/Courses/GraduateSummer/Shell/None
File Edit View Search Terminal Help
shahriar@earth:~/Teaching/Courses/GraduateSummer/Shell$ ls
Afkhami  None
shahriar@earth:~/Teaching/Courses/GraduateSummer/Shell$ cd None/
shahriar@earth:~/Teaching/Courses/GraduateSummer/Shell/None$ ls
Makefile
shahriar@earth:~/Teaching/Courses/GraduateSummer/Shell/None$ ls -alt
total 8
drwxrwxr-x 4 shahriar shahriar 4096 Jun 18 16:24 ..
drwxrwxr-x 2 shahriar shahriar 4096 Jun 18 16:24 .
-rw-rw-r-- 1 shahriar shahriar    0 Jun 18 16:24 Makefile
shahriar@earth:~/Teaching/Courses/GraduateSummer/Shell/None$
```

In bash:

- Output can be redirected using `>` and `<`.
- Commands can be combined using pipelines.
- The history command can be used to view and repeat previous operations, while tab completion can be used to save re-typing.
- Directories (or folders) are nested to organize information hierarchically.
- Use `grep` to find things in files, and `find` to find files themselves.
- Programs can be paused, run in the background, or run on remote machines.
- The shell has variables like any other program, and these can be used to control how it behaves.

In UNIX, all data is organized into files, that are in turn organized into directories. A tree-like structure, filesystem, is used for file management. The File System:

- The file system is the set of files and directories the computer can access
- Everything in a particular directory must have a unique name
- On Unix, the file system has a unique root directory called /
- Every other directory is a child of it, or a child of a child, etc.
- A path is a description of how to find something in a file system
- An absolute path describes a location from the root directory down
- A relative path describes how to find something from some other location

Here we also list basic commands (most shell commands have cryptic names):

pwd	print working directory
ls	listing
cd	changing working directory
.	current directory
..	parent directory: directory immediately above this one
mkdir	creating directory
cp	copy a file
nano	text editor
rm	delete a file
rmdir	delete a directory
mv	move or rename a file or directory
wc	word count
cat	catenate file

By convention, flags start with “-”, as in “-c” or “-l”: “ls -F”, “ls -a”, “ls -Fa”, and “ls -s”. The real strength of the shell is its ability to combine simple utilities in useful ways called pipes and filters:

>	redirect the output to a file
<	redirect the input to a file
	pipe/connect processes together
*	pattern-matching wildcard
?	matches any single character

Some useful commands:

- awk - Text processing language: Find and Replace text within file(s)

- `chmod` - Change access permissions
- `grep` - Search file(s) for lines that match a given pattern
- `history` - Command History
- `locate` - Find files - simple but fast
- `whereis` - Search the user's \$path, man pages and source files for a program
- `xargs` - Execute utility, passing constructed argument list(s)

Linux Process Control:

<code>top</code>	display Linux tasks
<code>kill</code>	terminate a process
<code>ps</code>	report a snapshot of the current processes
<code>ctl C</code>	kill a running job
<code>&</code>	run in the background
<code>jobs</code>	tell processes are currently running in the background
<code>fg</code>	brings the background job into the foreground
<code>ctl Z</code>	pause the job and return control to the shell (use <code>fg</code> or <code>bg</code> to resume)

Environment variables:

<code>HOME</code>	The current user's home directory
<code>HOSTNAME</code>	This computer's name
<code>HOSTTYPE</code>	What kind of computer this is
<code>PATH</code>	Where to look for programs
<code>PWD</code>	Present working directory
<code>SHELL</code>	What shell is being run
<code>TEMP</code>	Where to store temporary files
<code>USER</code>	The current user's ID

If we want to set some variables' values automatically every time we run a shell, we can put the command to do this in a file called `.bashrc` in our home directory. The `."` character at the front prevents `ls` from listing this file unless we specifically ask it to using `-a` — we normally don't want to worry about it — and the `"rc"` at the end is an abbreviation for "run control".

2.2 Integrated Development Environments (IDE) and Text Editors

An Integrated Development Environments (IDE) is a collection of applications used for source-code editing, compilation tools, a debugger, and a version control system, and sometimes a graphical user interface (GUI), and an object-oriented software development support. An IDE includes a text editor, sometimes more than a simple one with convenient features such as syntax highlighting, interfaces, and navigation tools. As a centralized environment, it also includes a version control systems, compiling and build tools, and debugging tools. Using an IDE improves workflow and allows an easier detection of errors. Overall using an IDE improves the efficiency code development. For applications in this book, multi-language IDEs are most suitable; the support for multiple programming languages and best suited for beginner programmers. Examples of this type of IDE is the Visual Studio IDE and Eclipse, which are known for their useful features, such as language plugin.

Text editors are used to edit an existing file or to create a new file from scratch to just read a text file. There are many text editors in Unix, including screen-oriented text editors such as `vi`, `nano`, and `vim`, to others with more advanced features. Modern text editors are easy to use and navigate, have find and replace, syntax highlighting, and are customizable. Sublime Text editor is one of the best code editor for a complete development environment. Emacs is the best code editor with a highly customizable environment. Emacs also integrates with GNU debugger (GDB) to provide an IDE interface, also supporting compiling, running, and testing programs, as well as shell commands.

2.3 Compilers and makefiles

There are two ways to translate a program into computer instructions i) **interpreting** or **compiling**. An interpreter reads a high-level program and translates and carry out commands line-by-line. A compiler is a program that translates a high-level program all at once, creating an executable file that can be run later. The high-level program is called **source code** (".c" and ".h" files for C codes), the translated program is called the **object code** (".o"), and the executed program is called the **executable**. A compiler reads the source code, translates it, builds an object file, and creates a file that is the executable. Consider the following "Hello, World." example:

hello.h:

```
void print_hello ();
```

hello.c:

```
#include <stdio.h>
#include "hello.h"

void print_hello () {
    printf("Hello, world.");
}
```

main.c:

```
int main () {
    hello ();
    return 0;
}
```

This program can be compiled using the GNU C compiler as follows:

```
$ gcc main.c hello.c
$ gcc -o hello.exe main.c hello.c
$ gcc -c main.c hello.c -o hello.o main.o hello.o
```

where the first line compiles and links into an executable called “a.out”, the second line is the same as the first line with an executable name “hello.exe”, and the third line translates the C code into object files and then the linker links the object files and creates the executable. The advantage of the latter is that if only one of the source codes is modified, only the updated file will be recompiled. This avoids unnecessarily compiling all the files which in the case of large projects can be inefficient. To link external libraries, for example, one can add the linking line:

```
$ gcc -c main.c hello.c -o hello.o main.o hello.o -lm
```

where `-l` option links the standard math library, `m`. Compilers accept many options, such as optimization options, the “`-O`” options.

Big programs have thousands of lines of code, organized into hundreds of source code files, with their corresponding header files. The make utility is a tool for managing and maintaining large codes. When only some of the source files are modified, but not all,

t make can automatically determine what needs to be recompiled. A file called “Makefile” (or “makefile”) which contains the dependencies to build a target, has a set of rules to determine which parts of a code needs recompilation. In a sense, Makefile is a way to automating repeated procedure (or other complex tasks with dependencies). A Makefile consists of dependency rules, macros and suffix rules. Here is a Makefile example of the “Hello, world.” program above:

```
# This is a comment line.
# CC is a macro for the compiler.
CC = gcc
# CFLAGS will be the options passed to the compiler.
CFLAGS = -c -Wall

all: exe

exe: main.o hello.o
    $(CC) main.o hello.o -o exe

main.o: main.c
    $(CC) $(CFLAGS) main.c

hello.o: hello.c
    $(CC) $(CFLAGS) hello.c

clean:
    rm -rf *.o
```

The make utility looks at any dependency file and if it has a modification time more recent than its target file, it then recreates the target. Let’s look at the Makefile above:

```
exe: main.o hello.o
<tab> $(CC) main.o hello.o -o exe
```

- A rule consists of a target, here `exe`, dependencies, here `main.o hello.o` and commands to build the target: `$(CC) main.o hello.o -o exe`.
- `<tab>` character cannot be replaced by spaces.
- A “target” is the name of a file (here the executable). or an action (for example, `clean`, here).
- “dependency” files are used as to build the target.

- Commands in a rule are shell commands to be executed.
- Typing `make` execute the first target in Makefile, here “exe”.
- “clear” here is a phony target; i.e. no dependencies, and just a list of commands.
- Macros allow to avoid repeating entries; for example, here: `CC = gcc`.

References: <https://www.gnu.org/software/make/>

2.4 Debugging

As codes become longer and longer, it becomes more and more complex, and therefore easier to introduce errors. Programming errors are called **bugs** and the process of finding the cause of them and correcting them is called **debugging**. Compiler can detect syntax error; a program cannot be compiled is syntactically incorrect. The error that does not show up until the program is run is called run-time error. Logical and semantic errors do not cause compilation error and run successfully, but it will produce the right outcome.

Debugging is the integral part of programming. The GNU debugger, GDB, can be used to debug programs compiled with the the GNU C compiler, gcc. The compilation must be done with the compiler option “-g”; e.g.

```
$ gcc -g -c -o hello.exe main.c hello.c
```

To start GDB, type:

```
$ gdb hello.exe
```

which will bring up a debugging session. There are a number of gdb commands that are useful for a debugging session:

r(un) [arglist]	Runs the program in GDB with optional argument list.
b(reak) [file:]function/line	Puts a breakpoint to stop the program when reached.
delete [file:]function/line	Removes the breakpoint at the given point.
c(ontinue)	Resumes execution of the program after it is stopped.
n(ext)	When stopped, runs the next line of code, stepping over functions.
s(tep)	When stopped, runs the next line of code, stepping into functions.
q(uit)	Exits GDB.
p(rint) expr	Prints out the given expression.
display var	Displays the given variable at every step of execution.
l(ist)	Lists source code.
help [command]	Gives help with a specified command.
bt	Gives a backtrace

2.5 Version control

Version control is a great tool for software development. It synchronizes files, keeps track of changes, undoes changes (think of unlimited undoing), and keeps the master copy of the file, amongst many other features. It also allows multiple users to work on the same project in parallel. All large scale coding projects use version control; it allows more robust software development and a more coordinated and organized teamwork and collaboration. Version control saves the data or files after making modifications, allowing reverting back and retrieve specific versions. Consistent version control allows programmers to test new algorithms without fearing breaking versions that work. The way it works is that it provides a trace of changes in case the programmer needs to access earlier revisions. A version control system provides:

- reverting to past versions
- comparing different versions
- pushing full version history to a repository
- pulling history back from the repository
- merging versions
- allowing multiple developers to work together:
 - to combine versions
 - to track who made changes
 - to work in parallel

– to share unfinished work

There are a number of automated version control systems, such as CVS or Subversion. We recommend here a modern system, Git, that do not need a centralized server to host the repository (the repository is the folder inside our project folder that keeps track of all the changes made to the files in our project and record that history over time). Merging tools on Git also make it easy for multiple developers to work on the same files simultaneously. Below we will review Git on the command line.

When using Git for the first time, things need to be configured. Below are a few Git commands for configurations (on a command line, Git commands are written as `git verb`, where verb is what we actually want to do):

- `git config -global user.name "your name"`
- `git config -global user.email "your e-mail"`
- `git config -global core.autocrlf input` (For macOS and Linux)
- `git config -global core.editor "emacs"`
- `git config -global core.editor "Vim"`
- `git config -list`
- `git -help`

Once Git is configured, we can start using it by creating a repository first. Also note that the created repository automatically includes subdirectories (and their files). When for example we add a file to the repository, Git will indicate that a new file is created. Before telling Git to track the file, using `git add "filename"`, the

`git status` command will show "untracked files" message. Finally, to have Git record the changes, a "commit" must be run, using `git commit` command.

git init	Make the working directory a repository
git status	Show the status of the project
rm -rf .git	Remove the entire Git history of a project
git add <i>file</i>	Add file contents to the index
git commit	Record changes to the repository
git pull	Fetch from a remote repo and try to merge into the current branch
git push	Push new branches and data to a remote repository
git log	Show commit logs
git diff	Show changes between commits
git diff HEAD	Show the difference between the last commit
git diff HEAD~1	Show the difference between one before HEAD
git checkout	Check out a branch or paths to the working tree
git branch	List, create, or delete branches
gitk	Show GUI history
git clone url [dir]	Clone a remote repo to dir

For example, consider the previous example, `main.c`, `hello.c`, `hello.h` and the `Makefile`. To create a new `git` repository, type:

```
$ git init /PATH/TO/CODE/DIRECTORY
```

which will create the repository including the file `.git`. That directory should only be accessed through `git`. Next add the files to the repository, assuming the working directory is `/PATH/TO/CODE/DIRECTORY`:

```
$ git add main.c hello.c hello.h Makefile
```

Typing

```
$ git status
```

will show something like:

```
git — -bash — 75x20
Shahriars-MacBook-Pro-2:git shahriar$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)

    new file:   Makefile
    new file:   hello.c
    new file:   hello.h
    new file:   main.c

Shahriars-MacBook-Pro-2:git shahriar$
```

Typing:

```
$ git commit -m "commit message" main.c hello.c hello.h Makefile
```

will make these files part of the repository, with proper information included as the message. Now suppose that `main.c` is modified and updated. Then `git status` will show that the changes aren't committed yet. However, if it turns out that the changes will break the code, you can return to the previous version by typing:

```
$ git checkout -- main.c
```